WUJI HAND 2 · PEN SPINNING (转笔) · PROJECT WALKTHROUGH

The penspin-graph project, explained step by step

Each of the 11 stages: what it does, the formula behind it, and what Wuji's official method does instead.

Sources: penspin-graph repo (`src/penspin_graph/` , measured results in `runs/full3/`) · wuji-mjlab v2026.10.22 (`src/wuji_mjlab/tasks/pen_spin/` , `deploy/pen_spin/`). All results are from simulation.

❓ The question in plain words

Wuji gives 40 recorded pen-spinning clips and a policy (策略) that follows one clip at a time. How did this project turn that into a controller that starts from any hand state and keeps the pen spinning? And at every stage, what differs from Wuji's official method?

🎯 Where we will end up

1. Every frame becomes a point: $\mathbf{f}_t = \mathbf{w} \odot (\mathbf{x}_t - \boldsymbol{\mu})/\boldsymbol{\sigma} \in \mathbb{R}^{67}$.
2. Frames are linked inside each clip. New jumps $u \to v$ are added when $C(u,v) = \frac{1}{\delta}\sum_j \lambda_j \|\mathbf{f}_{u+1+j} - \mathbf{f}_{v+j}\| \le 6$, or when a clip's final hold can blend into another clip's start.
3. A jump is kept only if it is physically gentle (blend $b = \mathrm{clip}(\lceil \Delta q_{\max}/(\omega_{\max}\Delta t) - 1 \rceil, 2, 10)$) and if Wuji's own policy, $\mathbf{q}^{\mathrm{target}} = \mathbf{q}^{\mathrm{ref}} + \mathrm{EMA}(\mathrm{clip}(0.5\,\mathbf{a}))$, executes it in simulation about as well as the original clip.
4. 38,624 jumps pass. They connect 15,215 frames (59%) into a core the hand never has to leave.
5. From any state: nearest frame $v^* = \arg\min_v \|\mathbf{f}(s) - \mathbf{f}_v\|$, shortest path into a verified loop, then circle it.

same Wuji policy, same simulator scene	Wuji official: one clip	this project: stitched loop	gain
longest continuous spinning	18.0 s	**84.7 s**	4.7×
pen-shaft rotation in one run	1,476°	**8,709° (24 turns)**	5.9×
from 20 random states, 30 s: still spinning after 20 s	2 / 20 (no graph)	**8 / 20**	4×

Flow at a glance 流程总览

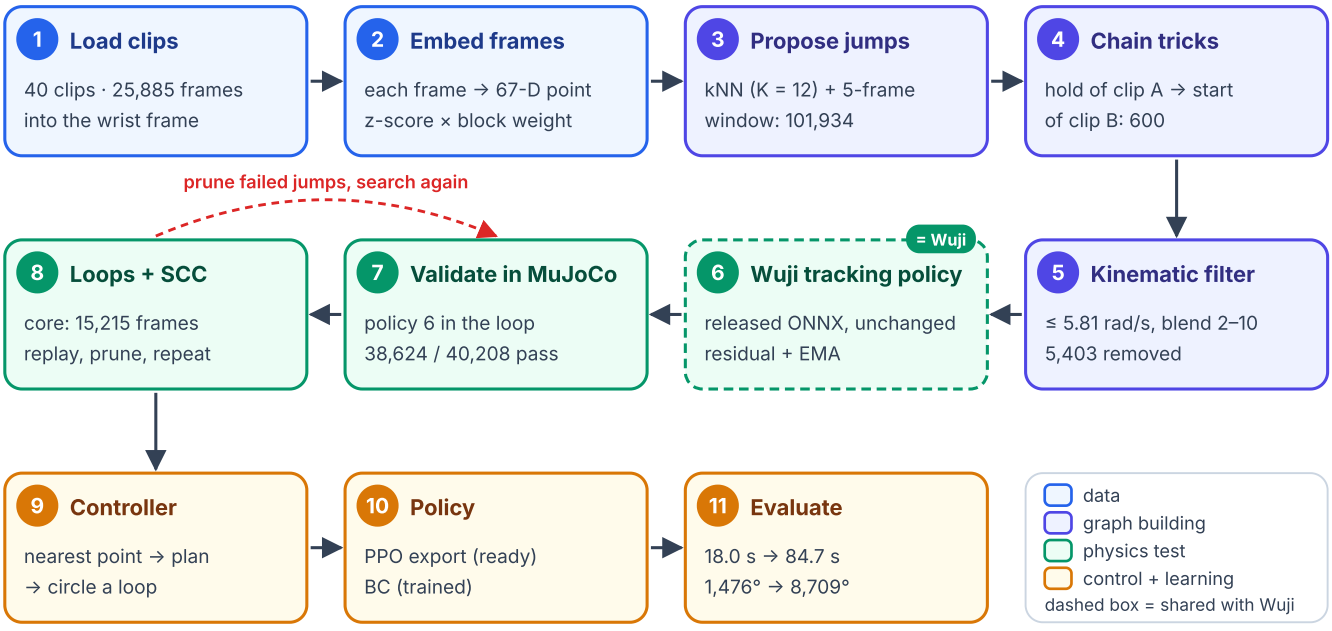


Figure 1. The 11 stages, numbered in order from 1 to 11. Stage 6 (dashed) is Wuji's released policy, reused unchanged. Red arc: a jump that breaks a long loop is rejected, then loops are searched again.

The 11 stages side by side with Wuji's method 逐步对比总览

red = the part that changes in this step R3 = rule from the toolbox (§0)

HIDDEN STEP = the code does it without saying FIX = a correction to an earlier message

SAME CHANGED NEW = this stage vs Wuji's official method



Figure 2. The same 11 stages side by side with Wuji's official pipeline. Dashed grey boxes mean "Wuji has no such step". The chip in the middle says how our stage relates to Wuji's: SAME reused, CHANGED same purpose but a different method, NEW no counterpart.

0 Symbols, rules and abbreviations 符号、规则与缩写

Symbols table

Symbol	Size	Meaning	Known?
t, T	scalar	frame index inside a clip; number of frames. Frames are $\Delta t = 0.02$ s apart (50 Hz).	✅ data
$\mathbf{q}_t$, $\dot{\mathbf{q}}_t$	20, 20	finger joint angles (关节角) and joint velocities (4 joints × 5 fingers)	✅ data
$\mathbf{p}_t$, $\boldsymbol{\rho}_t$	3, 4	pen centre position and pen orientation quaternion (四元数), in the wrist frame	✅ data
$\hat{\mathbf{n}}_t$	3	unit direction of the pen shaft (笔杆方向)	✅ from $\boldsymbol{\rho}_t$
$\boldsymbol{\omega}_t$	3	pen angular velocity (角速度), pen frame	✅ from $\boldsymbol{\rho}_t$
$\mathbf{l}_t$	21×3	palm + 20 finger-link positions; "tips" = the 5 distal links (15 numbers)	✅ data
s_t	0/1	resting flag: hand and pen are not moving	✅ computed once
$\mathbf{x}_t$, $\mathbf{f}_t$	67, 67	raw feature vector; normalised point (嵌入点)	✅ computed once
$\boldsymbol{\mu}$, $\boldsymbol{\sigma}$, w_b	67, 67, 7	mean and std of each feature over all frames; weight of block b	✅ fitted / chosen
δ	scalar	step unit (步长单位): median distance between consecutive frames = 0.565	✅ computed once
u, v	node ids	two frames (graph nodes); a jump $u \to v$ means "after u, play v"	❌ searched
K, W, λ_j	12, 5, 5	neighbours per query; window length in frames; window weights	✅ chosen
$C(u, v)$	scalar	window cost of a jump, in step units	❌ computed per jump
b, γ_i	int, scalar	number of blended frames at a jump; blend fraction $i/(b+1)$	❌ computed per jump
$\Delta q_{\max}$, $\omega_{\max}$	scalar	largest joint gap across a jump; joint-speed limit (5.81 rad/s)	❌ per jump / ✅ fitted
$\mathbf{o}_k$, $\mathbf{a}_k$	155, 20	policy observation and raw policy output at control tick k	❌ live
$\mathbf{r}_k$, $\bar{\mathbf{r}}_k$	20, 20	residual (残差) after scale + clip; residual after EMA smoothing	❌ live
$\mathbf{q}_k^{\text{ref}}$, $\mathbf{q}_k^{\text{target}}$	20, 20	reference joint angles fed to the policy; joint target sent to the hand	❌ live
e_k^{pos}, e_k^{ax}	scalar	pen position error (m) and shaft-direction error (rad) vs the reference	❌ measured
H	int	frames followed after a jump in the validation test (40, or 50 for hold jumps)	✅ chosen
$c(e)$	scalar	edge cost (边权) used by the shortest-path search	✅ formula
Φ	degrees	accumulated change of pen orientation, incl. roll (姿态累计变化); used to rank loops	❌ computed per loop
Θ	degrees	accumulated rotation of the pen shaft only (笔杆累计转角); used to evaluate	❌ measured

⚠ Look-alike symbols

- $\mathbf{q}$ is always joint angles. Quaternions are $\boldsymbol{\rho}$. (The code calls both "q".)
- v is a frame (node). The speed limit is $\omega_{\max}$, not "$v_{\max}$".
- δ (step unit, a feature distance) ≠ Δt (0.02 s) ≠ $\Delta q_{\max}$ (a joint gap in rad).
- $C(u, v)$ = how different two continuations are. $c(e) = 25 + 10\,C$ = what an edge costs the planner. They are related but not the same.
- $K = 12$ neighbours ≠ k = control tick.

Rules toolbox

#	Rule	Formula	1D version / note
R1	Point into a moving frame (坐标变换)	$\mathbf{p}^{\text{local}} = R^{\top}(\mathbf{p}^{\text{world}} - \mathbf{t})$	$x^{\text{local}} = x - t$
R2	Rotation into a frame; rotate a vector	$\boldsymbol{\rho}^{\text{local}} = \boldsymbol{\rho}_{\text{frame}}^{-1} \otimes \boldsymbol{\rho}^{\text{world}}$; $\hat{\mathbf{n}} = R(\boldsymbol{\rho})\,\mathbf{e}_x$	angles: $\theta^{\text{local}} = \theta - \theta_{\text{frame}}$
R3	z-score (标准化)	$z = (x - \mu)/\sigma$	$\mu = 0.4,\ \sigma = 0.2,\ x = 0.6 \Rightarrow z = 1$
R4	Weighted distance: the weight enters squared	$\lVert\mathbf{w} \odot \Delta\mathbf{z}\rVert^2 = \sum_i w_i^2\,\Delta z_i^2$	$w = 2$ counts $4\times$
R5	Finite difference (有限差分)	$\dot{x}_t \approx (x_{t+1} - x_t)/\Delta t$	$0.40 \to 0.43$ rad: 1.5 rad/s
R6	Median as a robust unit (中位数)	median ignores a few huge or tiny values	holds give tiny steps; flicks give huge ones
R7	Normalised weights	$\lambda_j = d^j / \sum_i d^i \ \Rightarrow\ \sum_j \lambda_j = 1$	a weighted average
R8	Smallest integer not below x (向上取整)	$n \geq x,\ n \in \mathbb{Z} \iff n \geq \lceil x \rceil$	$\lceil 3.3 \rceil = 4$
R9	Lerp and slerp (线性 / 球面插值)	$(1-\gamma)\,\mathbf{a} + \gamma\,\mathbf{b}$; slerp keeps a quaternion unit-length	$\gamma = 0.25$: a quarter of the way
R10	Angle between unit vectors	$\theta = \arccos(\hat{\mathbf{a}} \cdot \hat{\mathbf{b}})$, dot product clipped to $[-1, 1]$	$\cos 60^\circ = 0.5$
R11	Clip (截断)	$\text{clip}(x, l, h) = \min(\max(x, l), h)$	$\text{clip}(1.0, -0.6, 0.6) = 0.6$
R12	EMA (Exponential Moving Average, 指数滑动平均)	$\bar{r}_k = \alpha r_k + (1-\alpha)\bar{r}_{k-1}$; for constant r: $\bar{r}_k = r\,(1 - (1-\alpha)^k)$	$\alpha = 0.5$: 0.3, 0.45, ... → 0.6
R13	Dijkstra shortest path (迪杰斯特拉最短路径)	cheapest path when all edge costs are ≥ 0	like road navigation
R14	SCC (Strongly Connected Component, 强连通分量)	u, v in one SCC $\iff u \rightsquigarrow v$ and $v \rightsquigarrow u$	a "roundabout" region
R15	kNN (k-Nearest Neighbours, k近邻)	the K points closest to a query	a sorted distance list, top K
R16	Soft bound with tanh (双曲正切限幅)	$\beta(x) = c\tanh(x/c)$: $\approx x$ when $\lvert x\rvert \ll c$, never beyond $\pm c$	$c = 0.15$: $0.03 \to 0.0296$
R17	Angle between two orientations (姿态夹角)	$\angle(\boldsymbol{\rho}_a, \boldsymbol{\rho}_b) = 2\arccos\lvert\langle\boldsymbol{\rho}_a, \boldsymbol{\rho}_b\rangle\rvert$	includes roll about the shaft

Abbreviations used in this document

Short	English full name	中文全称
PPO	Proximal Policy Optimization	近端策略优化
RL	Reinforcement Learning	强化学习
BC	Behavior Cloning	行为克隆

DAgger	Dataset Aggregation	数据集聚合
DR	Domain Randomization	域随机化
EMA	Exponential Moving Average	指数滑动平均
kNN	k-Nearest Neighbours	k近邻
SCC	Strongly Connected Component	强连通分量
MLP	Multi-Layer Perceptron	多层感知机
MJCF	MuJoCo XML Format (MuJoCo = Multi-Joint dynamics with Contact)	MuJoCo 模型文件格式（多关节接触动力学）
ONNX	Open Neural Network Exchange	开放神经网络交换格式
NPZ	NumPy Zipped archive	NumPy 压缩数组文件
PD	Proportional-Derivative (controller)	比例-微分（控制器）
CPU / GPU	Central / Graphics Processing Unit	中央处理器 / 图形处理器
ELU	Exponential Linear Unit	指数线性单元
SHA-256	Secure Hash Algorithm, 256-bit	256位安全散列算法

Coder's view: where each symbol lives in the code

Symbol	Code name	File
$\mathbf{f}_t$, δ	`graph.features[t]`, `graph.step_unit`	`graph/motion_graph.py`
s_t	`graph.static_mask`, `graph.active_end`	`graph/motion_graph.py`
$C(u,v)$, b	`graph.jumps.cost`, `graph.jumps.blend`	`graph/motion_graph.py`, `graph/kinematics.py`
$\omega_{\max}$	`KinematicLimits.max_joint_vel`	`graph/kinematics.py`
0.5, ±0.6, α	`action_scale`, `residual_clip`, `ema_alpha`	`sim/controller.py`
H, 1.5	`follow_frames`, `relative_factor`	`sim/validator.py`
$c(e)$	`jump_cost_offset=25`, `jump_cost_scale=10`	`graph/loops.py`, `graph/planner.py`

1 Load the clips and move everything into the wrist frame 读取数据 · 手腕坐标系

✍️ **What the code writes** (src/penspin_graph/data/clips.py)

```
pen_pose   = pose_to_local(object_pose_w, wrist_pose_w)     # pen pose in the wrist frame
link_pos   = points_to_local(link_pos_w, wrist_pose_w)      # 21 link positions in the wrist frame
qvel       = finite_difference(qpos, frame_dt)              # joint velocities
pen_angvel = angular_velocity_from_quats(pen_pose[:, 3:], frame_dt)
```

Step 1.1 The raw data of one frame

$$\text{NPZ frame } t: \quad \mathbf{q}_t \in \mathbb{R}^{20}, \quad (\mathbf{t}_t^w,\ \boldsymbol{\rho}_t^w), \quad (\mathbf{p}_t^{\text{world}},\ \boldsymbol{\rho}_t^{\text{world}}), \quad \mathbf{l}_t^{\text{world}} \in \mathbb{R}^{21\times 3}$$

Why: This is what Wuji's 40 clips contain: joint angles, the wrist pose, the pen pose and the link positions. Everything except the joints is in the capture world frame. Frames are 0.02 s apart. In total: 5 motion groups, 25,885 frames (8.6 min).

Step 1.2 Pen position into the wrist frame R1

$$\mathbf{p}_t = R(\boldsymbol{\rho}_t^w)^\top \left(\mathbf{p}_t^{\text{world}} - \mathbf{t}_t^w\right)$$

Why: First subtract the wrist origin. Then undo the wrist rotation. After this, the pen position no longer depends on where the hand stood during the recording. The same rule moves the 21 link positions into the wrist frame.

Step 1.3 Pen orientation into the wrist frame R2

$$\boldsymbol{\rho}_t = (\boldsymbol{\rho}_t^w)^{-1} \otimes \boldsymbol{\rho}_t^{\text{world}}$$

Why: A quaternion is "re-based" by multiplying with the inverse of the frame's quaternion. Checked numerically: R1 and R2 reproduce the stored wrist-local arrays exactly.

Step 1.4 The shaft direction R2

$$\hat{\mathbf{n}}_t = R(\boldsymbol{\rho}_t)\,\mathbf{e}_x, \qquad \mathbf{e}_x = (1,0,0)^\top$$

Why: In the pen model the long axis is the pen's own x axis. For spinning, only where the shaft points matters. Roll about the shaft is invisible to the task, and Wuji's policy ignores it too.

Step 1.5 Velocities by finite differences R5 HIDDEN STEP

$$\dot{\mathbf{q}}_t = \frac{\mathbf{q}_{t+1} - \mathbf{q}_t}{\Delta t}, \qquad \boldsymbol{\omega}_t = \frac{\log\left(\boldsymbol{\rho}_t^{-1} \otimes \boldsymbol{\rho}_{t+1}\right)}{\Delta t}$$

Why: The clips store no velocities, so they are computed from the next frame. The pen's angular velocity is the rotation from frame t to $t+1$, written as a rotation vector, divided by Δt. Hidden detail: the last frame has no next frame, so the code copies the previous value.

Step 1.6 Mark resting frames

$$s_t = \left[\max_i |\dot{q}_{t,i}| < 0.2 \ \tfrac{\text{rad}}{\text{s}}\right] \wedge \left[\|\boldsymbol{\omega}_t\| < 0.3 \ \tfrac{\text{rad}}{\text{s}}\right]$$

Why: Every clip ends with a hold (静止段): the hand pinches the pen and nothing moves. The median hold lasts 2.0 s. The first frame of this trailing hold is called the clip's active end. Stage 3 keeps holds out of normal jumps. Stage 4 uses holds on purpose.

🔢 Example (1D)

Clip A: wrist at $x = 0.30$ m, pen at $x = 0.37$ m $\Rightarrow$ local 0.07 m. Clip B: wrist at $x = 0.50$ m, pen at $x = 0.57$ m $\Rightarrow$ local 0.07 m. In the wrist frame the two states are identical. In the world frame they look 20 cm apart. Velocity: a joint goes from 0.40 to 0.43 rad in one frame $\Rightarrow 0.03/0.02 = 1.5$ rad/s.

⇄ Wuji official vs this project SAME data CHANGED use

	Wuji official (wuji-mjlab)	This project (penspin-graph)
Data	40 clips in 5 groups at 50 Hz. Human hand motion retargeted (重定向) to the robot hand by Wuji.	The same files, unchanged (SHA-256 checked).
Frame of reference	Wrist-local. Every clip is re-based onto one fixed physical wrist mount.	The same wrist-local math.
What a frame is for	"Where the hand should be at time k" in one clip.	A standalone state that can be compared with any other frame.
Extra per-frame data	Not needed. The policy sees the reference frame directly.	Velocities, shaft direction, resting flag, active end.

2 Turn every frame into one 67-D point 嵌入 · 高维向量点

✍️ **What the code writes** (src/penspin_graph/features/featurizer.py)

```
for b in self.active_blocks:
    z = (raw[b] - self.mean[b]) / self.std[b]      # z-score per dimension
    parts.append(self.block_weights[b] * z)        # times the block weight
return np.concatenate(parts, axis=1)               # 67 numbers per frame
```

Step 2.1 Stack seven blocks

$$\mathbf{x}_t = \left[\ \mathbf{q}_t\ ;\ \dot{\mathbf{q}}_t\ ;\ \mathbf{p}_t\ ;\ \hat{\mathbf{n}}_t\ ;\ \dot{\mathbf{p}}_t\ ;\ \boldsymbol{\omega}_t\ ;\ \mathbf{l}_t^{\text{tips}}\ \right]$$

Why: These numbers describe what the hand and pen are doing right now. Shape check: $20+20+3+3+3+3+15=67$.

Step 2.2 z-score each dimension R3

$$z_{t,i} = \frac{x_{t,i} - \mu_i}{\sigma_i}, \qquad \mu_i, \sigma_i \text{ over all } 25{,}885 \text{ frames}, \quad \sigma_i \geq 10^{-3}$$

Why: The blocks have different units. Joint angles vary by 0.08–0.61 rad. Pen positions vary by only 1.4–2.1 cm. Without the z-score, a metre-valued block would vanish next to a radian-valued block.

Step 2.3 Multiply by a block weight

$$f_{t,i} = w_{b(i)}\, z_{t,i}, \qquad w = \{\mathbf{q}\colon 1,\ \dot{\mathbf{q}}\colon 0.4,\ \mathbf{p}\colon 2,\ \hat{\mathbf{n}}\colon 2,\ \dot{\mathbf{p}}\colon 0.4,\ \boldsymbol{\omega}\colon 0.4,\ \text{tips}\colon 1\}$$

Why: The weights say what "similar" means for pen spinning. The pen pose counts most, poses count normally, and velocities count a little.

Step 2.4 Distance between two frames R4

$$d(t,t') = \|\mathbf{f}_t - \mathbf{f}_{t'}\| = \sqrt{\textstyle\sum_b w_b^2\, \|\mathbf{z}_{t,b} - \mathbf{z}_{t',b}\|^2}$$

Why: The weight is squared inside the distance. So pen position, with weight 2, counts 4× per dimension compared with one joint angle.

Step 2.5 A natural unit: the step unit δ R6

$$\delta = \operatorname*{median}_t \|\mathbf{f}_{t+1} - \mathbf{f}_t\| = 0.565, \qquad \text{report every distance as } d/\delta$$

Why: $d/\delta = 1$ means "as different as one typical 50 Hz step". This makes every threshold readable. The median ignores the many tiny steps inside holds (10th percentile 0.08) and the few fast flicks (90th percentile 1.53).

🔢 **Example**

Joint: $\mu = 0.40$, $\sigma = 0.20$ rad. Frame A at 0.60 rad gives $z = 1.0$. Frame B at 0.50 rad gives $z = 0.5$.
Contribution to d^2: $1^2 \cdot 0.5^2 = 0.25$.
Pen x: $\sigma = 2$ cm. A at 7 cm, B at 9 cm, so $\Delta z = 1.0$. Contribution: $2^2 \cdot 1.0^2 = 4$.
So a 2 cm pen gap counts 16× more than a 0.1 rad joint gap.

💡 **Coder's view**

`f = w * (x - mu) / sigma` and `d = np.linalg.norm(f[a] - f[b]) / delta`. This is the same normalisation you would apply before kNN in any machine-learning pipeline. The only extra choice is the block weights.

⇄ Wuji official vs this project NEW

	Wuji official	This project
State representation	None for planning. The policy observes the raw reference frame: 89 numbers (20 joint angles, 6 pen axis-point coordinates, 63 link coordinates) plus the live state.	A normalised 67-D point for every frame.
Comparing frames	Never needed. Frame k of clip c is fixed by the clock.	Any two frames can be compared with d/δ.
Knobs	—	Block weights w_b (CLI option `--weights`).

3 Propose jumps: kNN + window matching 候选跳转 · k近邻 + 窗口匹配

✍ What the code writes (src/penspin_graph/graph/motion_graph.py)

```
q = F[u + 1]                                          # query: the frame we would play next
d, v = self.knn_filtered(q, k, exclude_episode=..., min_frame_gap=..., allowed=target_ok)
for w in range(W):                                    # compare the next W frames
    cost += weights[w] * np.linalg.norm(F[uu + 1 + w] - F[vv + w], axis=1)
cost /= self.step_unit
```

Step 3.1 What a jump means

$$\text{recorded: } u \to u+1 \to u+2 \to \cdots \qquad \text{jump: } u \to v \to v+1 \to \cdots$$

Why: A jump replaces "the rest of this clip" with "the rest of another clip (or another moment of this clip)". Frame v takes the place of $u+1$.

Step 3.2 Match the next frame, not the current one HIDDEN STEP

$$\text{want} \quad \mathbf{f}_v \approx \mathbf{f}_{u+1} \qquad (\text{not } \mathbf{f}_u)$$

Why: v is what the hand will be asked to do next. So it must look like what the hand would have done next, which is $u+1$. Your original idea said "find the most related point except its own $t+1$". This step makes it precise.

Step 3.3 Ask for the $K = 12$ nearest frames R15

$$\mathcal{V}_K(u) = \mathrm{KNN}_{12}\left(\mathbf{f}_{u+1}\right) \text{ restricted to allowed } v$$

Why: Allowed means: (a) both u and v are moving, not resting ($s = 0$); (b) v has at least 4 more frames after it; (c) v is in another clip, or in the same clip at least 50 frames (1 s) away from u.

Step 3.4 Window cost R7

$$C(u,v) = \frac{1}{\delta}\sum_{j=0}^{W-1} \lambda_j \left\|\mathbf{f}_{u+1+j} - \mathbf{f}_{v+j}\right\|, \qquad \lambda_j = \frac{0.8^{\,j}}{\sum_{i=0}^{W-1} 0.8^{\,i}}$$

Why: One similar frame is not enough. A pen moving up and a pen moving down can pass through the same pose. Comparing the next $W = 5$ frames (0.1 s) checks that both continuations are about to do the same thing. With $W = 5$: $\lambda = (0.298,\ 0.238,\ 0.190,\ 0.152,\ 0.122)$, so earlier frames count more because they are executed first.

Step 3.5 Keep the cheap ones

$$\text{keep } u \to v \quad \text{if} \quad C(u,v) \le 6$$

Why: 6 step units still allows clearly different clips to connect. Stages 5 and 7 then decide which of them the hand can really execute. Result: 101,934 candidates. Of all jumps that later pass simulation (stage 7), 26,803 cross between clips and 11,821 stay inside one clip.

🔢 Example

Distances over the window, in step units: $2, 2, 3, 3, 4$. $C = 0.298 \cdot 2 + 0.238 \cdot 2 + 0.190 \cdot 3 + 0.152 \cdot 3 + 0.122 \cdot 4 = 2.59 \le 6 \Rightarrow$ kept. Check on real data: recomputing C by hand for a stored jump gives 1.5997, the same as the stored value.

⚠ Two rules learned the hard way

- **Holds are excluded.** All resting frames look almost identical. Without this rule the cheapest "jumps" are hold → hold inside one clip. They cost about 0, always pass, and are useless.
- **Same-clip jumps need a 1 s gap.** Otherwise $u \to u + 2$ would count as a jump. A jump back in time inside a continuous-spin clip is the most natural way to repeat a turn.

⇄ Wuji official vs this project NEW

	Wuji official	This project
Edges between frames	Only $t \to t+1$ inside one clip.	Plus 101,934 proposed jumps, between and within clips.
Where motion can go next	Exactly one place.	Up to 12 alternatives per frame.
Clip boundary	Hard: the training episode ends at `reference_end`.	Soft: it can be crossed through a validated jump.
Where variety comes from	Training: a random start time inside a clip (adaptive 1 s bins weighted by past failures), plus a random pen slide of ±2 cm and roll of ±20° at reset.	New edges in the data. No training.

4 Chain tricks through the resting grasp 动作衔接 · 静止抓握

✍ What the code writes (MotionGraph.propose_rest_transitions)

```
# u = first hold frame of clip a,  v = first moving frame of clip b
u = self.node_id(a, min(int(self.active_end[a]), int(self.episode_lengths[a]) - 1))
v = self.node_id(b, self.first_moving_frame(b))
if dpos > max_pen_pos_m or dax > max_pen_axis_rad:
    continue                                                # pen too far apart
blend = int(max(blend_frames, np.ceil(dq / (max_joint_vel * dt) - 1.0)))   # slow blend
```

Step 4.1 The problem, measured

$$\underset{\text{clips}}{\text{median}} \left(\underset{t}{\text{median}} \min_{|t'-t| \ge 50} \frac{d(t,t')}{\delta} \right) = 10.6 \ \ (\text{range } 6.9\text{–}13.3) \ > \ 6$$

Why: Inside one clip, the closest frame at least 1 s away is usually more than 10 step units away. So the full state is not periodic: the fingers re-grasp differently on every turn. Stage 3 alone therefore produced only tiny loops. The largest region the hand could stay in forever was just 670 frames.

Step 4.2 **The state all clips share**

$$u_a = \text{first hold frame of clip } a, \qquad v_b = \text{first moving frame of clip } b \;\; (b \neq a)$$

Why: Every clip ends in a resting pinch grasp, and every clip starts from a similar calm grasp. This is the natural "hub" for chaining one trick into another.

Step 4.3 **The pen must already be close** R10

$$e^{\text{pos}} = \|\mathbf{p}_{u_a} - \mathbf{p}_{v_b}\| \leq 6 \text{ cm}, \qquad e^{\text{ax}} = \arccos\left(\hat{\mathbf{n}}_{u_a} \cdot \hat{\mathbf{n}}_{v_b}\right) \leq 60^\circ$$

Why: During the blend the fingers may re-shape the grasp, but the pen should not have to travel. The next clip's reference expects the pen where its first frame has it.

Step 4.4a **Speed during a blend of b frames**

$$\text{joint speed} = \frac{\Delta q_{\max}}{(b+1)\,\Delta t}, \qquad \Delta q_{\max} = \max_i |q_{v_b,i} - q_{u_a,i}|$$

Why: b inserted frames split the gap into $b+1$ equal steps of Δt each.

Step 4.4b **Require a slow speed (3 rad/s)**

$$\frac{\Delta q_{\max}}{(b+1)\,\Delta t} \leq 3 \;\tfrac{\text{rad}}{\text{s}}$$

Why: A hold-to-start blend re-grasps the pen, so it is done slowly on purpose. This is about half of the normal limit used in stage 5.

Step 4.4c **Solve for $b+1$**

$$b+1 \;\geq\; \frac{\Delta q_{\max}}{3\,\Delta t}$$

Why: Multiply both sides by $(b+1)/3$. It is positive, so the inequality keeps its direction: $\frac{\Delta q_{\max}}{3\Delta t} \leq b+1$. Read from right to left, that is the line above.

Step 4.4d **Solve for b, then take the smallest integer** R8

$$b \;\geq\; \frac{\Delta q_{\max}}{3\,\Delta t} - 1 \qquad \Longrightarrow \qquad b = \left\lceil \frac{\Delta q_{\max}}{3\,\Delta t} - 1 \right\rceil$$

Why: b must be a whole number of frames. The smallest whole number that is not below the bound is its ceiling.

Step 4.4e **Never shorter than 0.5 s, never longer than 1 s**

$$b \;\leftarrow\; \max\left(b,\; 25\right), \qquad \text{reject if } b > 50$$

Why: Even a small gap gets 25 frames, so the policy has time to settle the grasp. A gap that needs more than 50 frames is too far to bridge.

🔢 **Example**

$\Delta q_{\max} = 1.2$ rad: $1.2/(3 \cdot 0.02) - 1 = 19$, so $\lceil 19 \rceil = 19$ and $\max(19, 25) = 25$ frames (0.5 s). $\Delta q_{\max} = 2.4$ rad: $2.4/0.06 - 1 = 39$, so 39 frames (0.78 s).

Step 4.5 **What it gave**

$$600 \text{ proposed} \;\rightarrow\; 482 \text{ start in a real hold, tested} \;\rightarrow\; 99 \text{ pass} \;\; (98 \text{ after stage-8 pruning})$$

Why: About 100 extra edges grew the region where the hand can stay forever from 670 to 15,215 frames (59% of the data). Normal jumps give variety inside one trick. Hold transitions chain one trick into another, like dance moves linked through a neutral pose. The other 118 proposals come from 6 clips without a real hold at the end and were not tested.

⇄ Wuji official vs this project NEW

	Wuji official	This project
End of a clip	Training: `reference_end` ends the episode. Deploy: the clip clock stops at the last frame.	The hold is a hub. From it, 98 validated blends lead into other clips.
Switching to another trick	A new run, started from that clip's first frame.	Automatic, inside one run.
Transition motion	—	25–48 blended frames (0.5–1 s), at most 3 rad/s.

5 Kinematic filter and adaptive blend 运动学过滤 · 自适应插值

✍️ **What the code writes** (src/penspin_graph/graph/kinematics.py)

```
dq     = np.abs(q_v - q_u).max(axis=1)
needed = np.ceil(dq / (limits.max_joint_vel * dt) - 1.0).astype(int)
blend  = np.clip(np.maximum(needed, limits.blend_frames), limits.blend_frames, limits.max_blend_frames)
speed_ok = dq / ((blend + 1) * dt) <= limits.max_joint_vel + 1e-9
```

Step 5.1 A speed limit taken from the data

$$\omega_{\max} = 1.5 \times P_{99.9}\big(|\dot{q}|\big) = 1.5 \times 3.87 = 5.81 \ \tfrac{\text{rad}}{\text{s}}$$

Why: While spinning, the recorded hand moved at up to 3.87 rad/s (99.9th percentile, 第99.9百分位数). The factor 1.5 leaves room, but it still rejects jumps that would need a violent snap.

Step 5.2 The minimal blend (same algebra as 4.4a–d)

$$b_{\text{need}} = \left\lceil \frac{\Delta q_{\max}}{\omega_{\max}\,\Delta t} - 1 \right\rceil$$

Why: Exactly steps 4.4a–4.4d, with 3 rad/s replaced by $\omega_{\max}$.

Step 5.3 Clip to the allowed range R11

$$b = \text{clip}\,\big(b_{\text{need}},\ 2,\ 10\big)$$

Why: Every jump gets at least 2 smoothing frames. No jump may take more than 10 frames (0.2 s), because a long blend leaves the recorded motion for too long.

Step 5.4 Re-check the speed after clipping

$$\text{ok}_{\text{speed}} \iff \frac{\Delta q_{\max}}{(b+1)\,\Delta t} \le \omega_{\max}$$

Why: This can only fail when step 5.3 cut b down to 10. Then even the longest allowed blend is too fast, and the jump is dropped.

Step 5.5 Joint limits: checking the two ends is enough HIDDEN STEP

$$l = (1-\gamma)l + \gamma l \ \le\ (1-\gamma)\,q_u + \gamma\,q_v \ \le\ (1-\gamma)h + \gamma h = h, \qquad \gamma \in [0,1]$$

Why: If both end poses are inside the limits $[l, h]$, every straight-line blend between them is inside too. So the code checks only $\mathbf{q}_u$ and $\mathbf{q}_v$.

Step 5.6 The pen cannot teleport R10

$$e^{\text{pos}} = \|\mathbf{p}_{u+1} - \mathbf{p}_v\| \le \tau_{\text{pos}}(b), \qquad \tau_{\text{pos}}(b) = \min\big(2\,\text{cm}\cdot(1 + 0.5\,(b-2)),\ 6\,\text{cm}\big)$$
$$e^{\text{ax}} = \arccos\big(\hat{\mathbf{n}}_{u+1}\cdot\hat{\mathbf{n}}_v\big) \le \tau_{\text{ax}}(b), \qquad \tau_{\text{ax}}(b) = \min\big(20^\circ\cdot(1 + 0.5\,(b-2)),\ 45^\circ\big)$$

Why: The data says the pen will be at $\mathbf{p}_{u+1}$. The jump pretends it is at $\mathbf{p}_v$. The gap must be small. A longer blend gives the policy more time to move the pen, so the tolerance grows with b up to a cap.

Step 5.7 **Build the blended frames** R9

$$\gamma_i = \frac{i}{b+1}, \quad \mathbf{q}^{(i)} = (1-\gamma_i)\,\mathbf{q}_u + \gamma_i\,\mathbf{q}_v, \quad \mathbf{p}^{(i)} = (1-\gamma_i)\,\mathbf{p}_u + \gamma_i\,\mathbf{p}_v, \quad \boldsymbol{\rho}^{(i)} = \mathrm{slerp}(\boldsymbol{\rho}_u, \boldsymbol{\rho}_v, \gamma_i)$$

Why: For $i = 1, \ldots, b$. Joints, pen position and link positions are interpolated in a straight line. The pen quaternion uses slerp, so it stays a valid rotation.

🔢 **Example**

$\Delta q_{\max} = 0.5$ rad: $0.5/(5.81 \cdot 0.02) - 1 = 3.30$, so $b_{\text{need}} = \lceil 3.30 \rceil = 4$ and $b = \mathrm{clip}(4, 2, 10) = 4$. Speed: $0.5/(5 \cdot 0.02) = 5.0 \le 5.81$ ✓. Tolerances at $b = 4$: $\tau_{\text{pos}} = 2 \cdot (1 + 0.5 \cdot 2) = 4$ cm and $\tau_{\text{ax}} = 40°$. Blend fractions: $\gamma = 0.2, 0.4, 0.6, 0.8$.

Real data: 5,403 proposals removed. Among validated jumps, blends of 2/3/4/5/6 frames occur 31,025 / 5,348 / 1,854 / 289 / 10 times.

⇄ Wuji official vs this project NEW for new edges SAME action safety

	Wuji official	This project
Reference smoothness	Clips used as recorded. They are smooth because they were captured continuously.	Every jump point is smoothed with 2–10 blended frames (hold jumps: 25–50).
Speed and limit safety	In the action path: residual clipped to ±0.6 rad, EMA with α = 0.5, targets clamped to the joint limits. Training penalises action rate, torque and over-current.	The same action path (stage 6), plus explicit checks on every new transition before it is ever used.
Pen continuity	Guaranteed by the recording.	Must be checked: 2–6 cm and 20–45°, depending on b.

6 The shared part: Wuji's tracking policy 共用部分 · 官方跟踪策略

✍️ **What the code writes** (src/penspin_graph/sim/controller.py, copied from deploy/pen_spin)

```
raw = sess.run(["actions"], {"obs": obs.reshape(1, -1)})[0]                    # a_k
residual  = np.clip(raw * self.action_scale, *self.residual_clip)              # r_k
processed = self.ema_alpha * residual + (1.0 - self.ema_alpha) * self._prev_residual
return np.asarray(ref.qpos, np.float64) + processed                            # q_target
```

Step 6.1 **The observation (155 numbers)**

$$\mathbf{o}_k = \big[\ \underbrace{\mathbf{q}_k^{\text{ref}},\ \mathbf{e}_k^{\text{ref}},\ \mathbf{l}_k^{\text{ref}}}_{\text{command: } 20+6+63\,=\,89}\ ,\ \mathbf{q}_k,\ 0.25\,\dot{\mathbf{q}}_k,\ \mathbf{e}_k,\ \mathbf{a}_{k-1}\ \big] \in \mathbb{R}^{155}$$

Why: The policy sees the reference frame (the command), the live hand state, the live pen, and its own last output. Shape check: $89 + 20 + 20 + 6 + 20 = 155$, the same as Wuji's `obs_dim`.

Step 6.2 **The pen as two shaft points** R16

$$\mathbf{e}^{\pm} = \mathbf{p} \pm 0.10\,\hat{\mathbf{n}}, \qquad \text{each coordinate } \ \beta(x) = 0.15\tanh(x/0.15)$$

Why: Two points 10 cm along the shaft encode both position and direction in 6 numbers. The tanh keeps a lost pen from producing huge inputs: $0.03 \to 0.0296$ (almost unchanged), $0.12 \to 0.0996$ (squashed).

Step 6.3 **Raw action**

$$\mathbf{a}_k = \pi_\theta(\mathbf{o}_k) \in \mathbb{R}^{20}$$

Why: π_θ is an MLP (Multi-Layer Perceptron, 多层感知机) with hidden layers 1024-512-256, loaded from Wuji's ONNX (Open Neural Network Exchange, 开放神经网络交换格式) file. There is one file per motion group.

Step 6.4 **Scale and clip** R11

$$\mathbf{r}_k = \text{clip}\left(0.5\,\mathbf{a}_k,\ -0.6,\ 0.6\right)$$

Why: The policy is allowed to correct each joint by at most ±0.6 rad.

Step 6.5 **Smooth over time** R12

$$\bar{\mathbf{r}}_k = 0.5\,\mathbf{r}_k + 0.5\,\bar{\mathbf{r}}_{k-1}$$

Why: The EMA (Exponential Moving Average, 指数滑动平均) removes jitter between control ticks. That protects the real hand's motors.

Step 6.6 **The joint target**

$$\mathbf{q}_k^{\text{target}} = \mathbf{q}_k^{\text{ref}} + \bar{\mathbf{r}}_k$$

Why: The policy is a corrector around the reference. The hand's PD (Proportional-Derivative, 比例-微分) position actuators then track the target for 4 physics steps of 0.005 s, which gives 50 Hz control.

🔢 **Example (one joint)**

Reference 0.80 rad, raw output $a = 2.0$: $r = \text{clip}(1.0, -0.6, 0.6) = 0.6$; $\bar{r}_1 = 0.5 \cdot 0.6 + 0 = 0.30$; target $= 1.10$ rad. If a stays 2.0: $\bar{r}_2 = 0.5 \cdot 0.6 + 0.5 \cdot 0.30 = 0.45$, target 1.25 rad, and $\bar{r} \to 0.6$ in the limit.

💡 **Meaning: two consequences used later**

- **Any test of a reference must include the policy.** Open-loop replay (no correction) drops the pen within about 0.2 s, even on the original clips. With the policy, the pen stays within about 1.1 cm of the reference on average. → stage 7.
- **The live joints can sit far from the reference while everything is fine.** So "am I lost?" must be judged by the pen, not by the joints. → stage 9.

⇄ Wuji official vs this project SAME

	Wuji official	This project
Policy	Trained with PPO (Proximal Policy Optimization, 近端策略优化): 4,096 parallel environments, about 10,000 iterations, one policy per motion group.	The same 5 released ONNX policies, weights unchanged. No training at all.
Simulator	mjlab on MuJoCo Warp (GPU), with heavy DR (Domain Randomization, 域随机化): friction 0.35–1.4, pen mass 0.4–1.6×, PD gains, encoder bias, disturbance forces, observation noise.	A standalone MuJoCo scene on CPU with the same essentials: fixed wrist, 0.005 s × 4, implicitfast integrator, condim-4 contacts, 30 g pen. No randomization.
Rewards	Pen position (weight 2), pen axis points (8), link positions (1), joint positions (1); penalties for action rate, torque, over-current and self-collision.	Not needed, because nothing is trained.
Obs / action path	155-D observation; residual × 0.5, clip ±0.6, EMA 0.5.	Identical. Check: the policy completes 30 of its 40 own clips in our scene.
Reference it receives	Frame k of one clip.	The frame chosen by the graph controller (stage 9).

7 Test every jump in simulation 仿真验证 · 每条跳转单独测试

✍️ **What the code writes** (src/penspin_graph/sim/validator.py)

```
stitched = rollout(self.scene, self.controller, ref_jump, start, thresholds=cfg.thresholds)
baseline = rollout(self.scene, self.controller, ref_base, start, thresholds=cfg.thresholds)
allowed  = cfg.relative_factor * max(baseline.mean_pos_err(), cfg.relative_floor_m)
ok = (not stitched.terminated) and tail_ok and stitched.mean_pos_err() <= allowed
```

Step 7.1 One start state

$$\text{start} = \left(\mathbf{q}_u,\ \dot{\mathbf{q}}_u,\ \mathbf{p}_u,\ \boldsymbol{\rho}_u,\ \dot{\mathbf{p}}_u,\ \boldsymbol{\omega}_u\right)$$

Why: The hand and the pen are placed exactly as recorded at frame u. Both test runs start from this same state, so the comparison is fair.

Step 7.2 Two references from the same start

$$\mathcal{R}^J = \big[u,\ \underbrace{\star \cdots \star}_{b \text{ blended}},\ v,\ v+1,\ \ldots,\ v+H-1\big], \qquad \mathcal{R}^B = \big[u,\ u+1,\ \ldots,\ u+L\big]$$

Why: $\mathcal{R}^J$ takes the jump. $\mathcal{R}^B$ (baseline) continues the original clip for the same length L. $H = 40$ frames (0.8 s); hold jumps use 50.

Step 7.3 Errors after every tick R10

$$e_k^{\text{pos}} = \|\mathbf{p}_k^{\text{sim}} - \mathbf{p}_k^{\text{ref}}\|, \qquad e_k^{\text{ax}} = \arccos\left(\hat{\mathbf{n}}_k^{\text{sim}} \cdot \hat{\mathbf{n}}_k^{\text{ref}}\right)$$

Why: Only the pen is judged. The pen is the task. The joints are free to deviate (stage 6).

Step 7.4 Fail fast, with Wuji's own thresholds

$$\text{fail} \iff \exists\, k :\ e_k^{\text{pos}} > 5\ \text{cm} \ \vee\ e_k^{\text{ax}} \geq 60^\circ$$

Why: These are exactly the termination limits of Wuji's training environment. If Wuji would call it a failure, so do we.

Step 7.5 It must end well

$$\tfrac{1}{5}\textstyle\sum_{\text{last } 5} e_k^{\text{pos}} \leq 3\ \text{cm}, \qquad \tfrac{1}{5}\textstyle\sum_{\text{last } 5} e_k^{\text{ax}} \leq 30^\circ$$

Why: Surviving is not enough. At the end of the test the pen must be back on its reference, ready for whatever comes next.

Step 7.6 No worse than the original clip

$$\overline{e^{\text{pos}}}\left(\mathcal{R}^J\right) \ \leq\ 1.5 \cdot \max\left(\overline{e^{\text{pos}}}\left(\mathcal{R}^B\right),\ 1\ \text{cm}\right)$$

Why: The clips are retargeted human motion, so even they are not perfectly trackable. The fair question is: "is the jump about as executable as just continuing the clip?" The 1 cm floor stops a near-perfect baseline from making the test impossibly strict.

🔢 Example

Baseline mean error 0.8 cm → floor → 1.0 cm → allowed $1.5 \cdot 1.0 = 1.5$ cm. Stitched run: mean 1.2 cm, last-5 mean 1.6 cm, never above 5 cm → **pass**. Another stitched run hits 5.2 cm at tick 18 → **fail** ("terminated: pen position").

Real data: 40,208 jumps tested, **38,624 pass (96%)**, about 60 ms per jump on one CPU core. Hold jumps are harder: 482 tested, 99 pass. Most of their failures are the pen moving more than 5 cm (244).

⇄ Wuji official vs this project NEW

	Wuji official	This project
What is checked	The policy, during training, on the fixed clips.	Each new edge, with the frozen policy.
Pass / fail rule	The episode terminates at 5 cm / 60°. Success = reaching `reference_end`.	No termination, end within 3 cm / 30°, and at most 1.5× the baseline error.
Cost	GPU hours of PPO per motion group.	About 60 ms CPU per edge: 40,208 edges ≈ 1.5 h on 2 cores.
Robustness	Trained under DR and noise.	One nominal scene, no randomization. This is a limitation.

8 Loops, the spinnable core and end-to-end pruning

环路 · 强连通核心 · 端到端剪枝

✍ **What the code writes** (src/penspin_graph/graph/loops.py, sim/verify.py)

```
dist, pred = dijkstra(adj, directed=True, indices=v, return_predecessors=True, limit=limit)
back = _path_from_predecessors(pred, v, u)        # cheapest way from v back to u
loop = _loop_from_nodes(graph, back, jumps)        # ... closed by the jump u -> v
blamed = _blame_jump(graph, traj, res.termination_step, window=100)   # after a failed replay
```

Step 8.1 The graph and its edge costs

$$G = (V, E), \qquad c(t \to t+1) = 1, \qquad c(u \to v) = 25 + 10\,C(u, v)$$

Why: V = the 25,885 frames. E = recorded steps plus validated jumps. With these costs a path's cost is roughly its number of frames, and every jump "costs" like 25 + 10C frames. So the search prefers to stay on recorded motion and jumps only when it must. Example: $C = 2$ costs like 45 frames.

Step 8.2 Close a loop through a jump R13

$$\text{loop}(u \to v) = \underbrace{(v \rightsquigarrow u)}_{\text{Dijkstra}} + (u \to v)$$

Why: A single clip is a chain, so every loop needs at least one jump. For a jump $u \to v$, the cheapest path from v back to u, followed by the jump itself, is the cheapest loop that uses this jump.

Step 8.3 Skip jumps that can never be on a loop R14 HIDDEN STEP

$$u \to v \text{ is on a loop} \iff \text{SCC}(u) = \text{SCC}(v) \text{ and } |\text{SCC}(u)| > 1$$

Why: (⇒) If a loop contains both, each reaches the other, so they are in one SCC. (⇐) If they are in one SCC, a path $v \rightsquigarrow u$ exists, and adding the edge $u \to v$ closes a cycle. Computing SCCs once removes most Dijkstra calls.

Step 8.4 Score a loop by how much the pen turns R17

$$\Phi = \sum_t \angle(\boldsymbol{\rho}_t, \boldsymbol{\rho}_{t+1}), \qquad \text{keep } \Phi \geq 180^\circ, \text{ rank by } \Phi/T$$

Why: The loop search sums the small change of pen orientation between consecutive frames. This favours loops where the pen keeps moving. Φ includes roll about the shaft, so it is a generous score. Stage 11 reports the stricter shaft-only rotation Θ instead.

Step 8.5 Replay whole loops and prune what breaks them

$$\text{replay fails at tick } k^* \;\Rightarrow\; \text{reject the last jump in } \left[k^* - 100,\ k^*\right] \;\Rightarrow\; \text{search again}$$

Why: Each jump passed a 0.8 s local test. But small errors can add up over a 15 s loop. The local test is necessary, not sufficient. So the top 30 loops are replayed in full, the guilty jump is removed, and loops are searched again. This ran for 5 rounds.

🔢 **Result**

Largest SCC: **15,215 frames (59%)**, from which the hand never has to stop. 151 loops with Φ ≥ 180°. 16 of the top 30 complete a full lap end to end.
Best loop: 752 recorded frames plus blends = **16.4 s per lap**, 9 jumps, 3 clips (`02/clip_003`, `03/clip_002`, `03/clip_003`). Shaft rotation Θ = **1,709° per lap ≈ 104°/s**. Its jump edges cost 35–196, so the planner pays for 9 jumps but gains a loop that never ends.

FIX **Rotation per lap and speed**

An earlier message said 2,360° per lap. That is Φ, the full orientation change including roll about the shaft. Shaft-only rotation, which is what spinning means, is 1,709° per lap. It also said 114°/s. That divided by 15.0 s of recorded frames and forgot the blended frames. A lap really takes 16.4 s, so the speed is about 104°/s (103°/s measured over the 84.7 s run).

⇄ Wuji official vs this project NEW

	Wuji official	This project
Cycles	None. Every clip is a chain that ends in a hold.	151 loops (Φ ≥ 180°) and 21 non-trivial SCCs.
Longest motion	One clip: at most 21.8 s of motion in the data. The longest clip the policy completes in our scene: 18.0 s.	A loop can repeat without end: 84.7 s tested (stage 11).
Checking a whole motion	Training on that clip.	A full replay plus pruning of the blamed jump.

9 The controller: nearest point → plan → circle 闭环控制 · 最近点 → 规划 → 绕环

✍️ **What the code writes** (src/penspin_graph/graph/planner.py, MotionMatchingController.step)

```
lost = self._pen_lost(state)                        # pen > 3 cm or > 40 deg off its reference
if self._plan and lost: self._plan = []             # drop the plan
d, idx = self._nearest(state)                       # nearest frame (prefer the core)
if not self._plan and self.goal_loop is not None:
    self._plan = self._plan_loop(node)              # Dijkstra into the loop, then one lap
```

Step 9.1 Measure and embed the live state

$$s_k = \left(\mathbf{q}_k, \dot{\mathbf{q}}_k, \mathbf{p}_k, \boldsymbol{\rho}_k, \dots\right) \longmapsto \mathbf{f}(s_k) \quad \left(\text{same } \boldsymbol{\mu}, \boldsymbol{\sigma}, \mathbf{w} \text{ as stage 2}\right)$$

Why: The live state goes through the same featurizer as the data, so it can be compared with every recorded frame. On the real hand the pen pose comes from the camera, the same signal Wuji's deploy code uses.

Step 9.2 Find the nearest recorded frame R15

$$v^* = \arg\min_v \|\mathbf{f}(s_k) - \mathbf{f}_v\|, \qquad \text{use } v^*_{\text{core}} \text{ if } d\left(s_k, v^*_{\text{core}}\right) \le d\left(s_k, v^*\right) + 2\delta$$

Why: This is your original "find the closest point" step. If a frame inside the core (the 59% SCC) is almost as close, it is preferred, because from the core there is always a way to keep going.

Step 9.3 Plan into the loop, then circle it R13

$$\text{plan} = \text{Dijkstra}\left(v^{*} \rightsquigarrow \mathcal{L}\right), \quad \text{then } \mathcal{L},\ \mathcal{L},\ \mathcal{L}, \ldots$$

Why: The path enters the best verified loop $\mathcal{L}$ at its cheapest frame. After that the controller replays the loop lap after lap, starting from the entry frame.

Step 9.4 Feed the plan to Wuji's policy

$$\mathbf{q}_k^{\text{ref}} = \mathbf{q}\big(\text{plan}[k]\big) \ \ (\text{blended at jumps}) \quad \Longrightarrow \quad \mathbf{q}_k^{\text{target}} = \mathbf{q}_k^{\text{ref}} + \bar{\mathbf{r}}_k \quad (\text{stage } 6)$$

Why: The planner only decides which frame to track next. The low-level work is exactly Wuji's policy.

Step 9.5 Re-match only when the pen is lost

$$\text{lost} \iff \|\mathbf{p}_k^{\text{sim}} - \mathbf{p}_k^{\text{ref}}\| > 3\ \text{cm} \ \vee \ \arccos\left(\hat{\mathbf{n}}_k^{\text{sim}} \cdot \hat{\mathbf{n}}_k^{\text{ref}}\right) > 40^{\circ}$$

Why: If lost: drop the plan, go back to 9.2 and 9.3. The first version re-matched on the full 67-D distance. It said "lost" almost every tick, because the policy keeps the joints up to 0.6 rad off the reference by design (stage 6), and it kept switching frames for no reason. Judging by the pen fixed this.

Step 9.6 Without a goal loop: escape before a clip ends

$$\begin{aligned} &\text{if} \quad \text{active frames left} < 250 \ \wedge \ \text{ticks since the last jump} \geq 50 \\ &\text{then} \quad \text{Dijkstra} \rightarrow \text{a frame with} \geq 250 \text{ active frames ahead} \end{aligned}$$

Why: This is setting C in stage 11. It keeps the hand moving through validated jumps but does not aim for a particular loop.

🔢 Example

Pen 2 cm and 25° off its reference → not lost → keep following the plan. Pen 4 cm off → lost → nearest frame again → new path into the loop.

⇄ Wuji official vs this project CHANGED

	Wuji official deploy (deploy/pen_spin)	This project
Which frame to track	$k = \text{round}\left((t_{\text{now}} - t_0)/\Delta t\right)$: wall-clock time since start.	Chosen from the measured state: nearest frame + plan.
Start	Hold on the clip's first frame, then the operator starts the clip.	Any state.
End	The clip clock stops at the last frame (done, or hold the last frame).	Loops never end.
After a disturbance	Keeps following the clock.	Re-matches to the nearest frame and re-plans.
Pen input	Camera-tracked marker tags → wrist-local pen pose.	The same signal (from simulation here).

10 From the graph to a policy 从运动图到策略

Two routes

Route A, export for Wuji's PPO. Loops and walks on the validated graph are written as a new clip group, `06_graph_stitched` (30 clips, 12.3 min). It uses Wuji's own clip format with SHA-256 digests and a recipe file, so Wuji's training command works unchanged. Status: ready, not trained yet (it needs the GPU).
Route B, BC (Behavior Cloning, 行为克隆). Copy what "graph + Wuji policy" does into one network that needs no reference. Steps 10.1–10.4 below.

✍ **What the code writes** (src/penspin_graph/policy/bc.py)

```
obs_all.append(bc_observation(prev.qpos, prev.qvel, prev.pen_pose))   # 46 numbers, no reference
res_all.append(res.targets[k] - prev.qpos)                            # what the expert commanded
loss = torch.nn.functional.mse_loss(model(X[b]), Y[b])
```

Step 10.1 Collect expert data

$$\mathbf{o}_k^{\mathrm{BC}} = \left[\mathbf{q}_k,\ 0.25\,\dot{\mathbf{q}}_k,\ \mathbf{e}_k\right] \in \mathbb{R}^{46}, \qquad \Delta\mathbf{q}_k = \mathbf{q}_k^{\mathrm{target}} - \mathbf{q}_k$$

Why: 200 random walks on the graph are executed with Wuji's policy. Each tick stores what the robot could measure and what the expert commanded. Shape check: $20 + 20 + 6 = 46$. The reference is *not* in the observation, and that is the point. Result: 30,323 samples.

Step 10.2 Normalise inputs and outputs R3

$$\hat{\mathbf{o}} = \frac{\mathbf{o} - \boldsymbol{\mu}_o}{\boldsymbol{\sigma}_o}, \qquad \widehat{\Delta\mathbf{q}} = \frac{\Delta\mathbf{q}}{s_{\max}}, \quad s_{\max} = \max|\Delta q|$$

Why: The same z-score idea as stage 2. The outputs are scaled into $[-1, 1]$ so the network trains well.

Step 10.3 Fit by least squares

$$\min_{\phi} \frac{1}{N}\sum_{k=1}^{N} \left\| g_\phi(\hat{\mathbf{o}}_k) - \widehat{\Delta\mathbf{q}_k} \right\|^2$$

Why: This is regression: input $\hat{\mathbf{o}}$, target $\widehat{\Delta\mathbf{q}}$. g_ϕ is an MLP 512-512-256 with ELU (Exponential Linear Unit, 指数线性单元) activations, trained for 40 epochs.

Step 10.4 Run it

$$\mathbf{q}_k^{\mathrm{target}} = \mathbf{q}_k + s_{\max}\, g_\phi(\hat{\mathbf{o}}_k)$$

Why: Undo the output scaling and add the result to the current joints. The network is also exported as an ONNX file.

⚠ **Result and why it is weaker**

BC survives 3.6 s on average and drops the pen in 55% of runs. Graph + Wuji policy survives 27–29 s (stage 11). This is expected: plain BC only sees expert states. A small error leads to states it never saw (covariate shift, 协变量偏移). The next step is DAgger (Dataset Aggregation, 数据集聚合), with graph + Wuji policy as the expert, or PPO on the route-A clips.

⇄ Wuji official vs this project CHANGED

	Wuji official	This project
How the policy is learned	PPO with rewards, from scratch, one policy per motion group.	(A) The same PPO, trained on stitched references that are already validated. (B) Imitation of graph + Wuji policy.
Reference at run time	Needed.	(A) Needed. (B) Not needed.
Compute	GPU, about 10,000 iterations.	(A) The same. (B) CPU, minutes.

11 Evaluation: original vs stitched 效果对比

Step 11.1 The metrics

$$\Theta = \sum_{k} \arccos\left(\hat{\mathbf{n}}_k \cdot \hat{\mathbf{n}}_{k+1}\right), \qquad \text{still spinning} \iff \Theta_{\text{last } 10\text{ s}} > 90^\circ$$

Why: Θ counts only the shaft direction, so it is stricter than the loop score Φ of step 8.4. "Survival" ends when the pen falls 12 cm below its start height, or when the tracking error trips Wuji's 5 cm / 60° limits.

⚠ How each number was measured

For the original clips and for the long loop run, Θ is the reference's shaft rotation over the span the policy tracked successfully. During that span the pen stays within 5 cm / 60° of the reference, about 1.1 cm on average. For the random starts, Θ is measured on the simulated pen itself. Everything uses the same scene and Wuji's same policy.

Endurance: Wuji's policy on its own clips vs the same policy on our best loop

metric	Wuji official: own clips	this project: best loop	gain
clips completed in our scene	30 / 40	—	
longest continuous spinning	18.0 s (`03/clip_007`)	**84.7 s** (5.2 laps)	4.7×
shaft rotation in one run	1,476°	**8,709°** (24 turns)	5.9×
average rotation speed	53°/s (mean over clips), 108°/s (fastest clip)	**103°/s**, sustained for 84.7 s	≈ 2× the mean; about the fastest clip's speed

From 20 random moving states, 30 s each (all with Wuji's policy at the low level)

setting	mean Θ	Θ in last 10 s	still spinning	drops	jumps / run
B: nearest frame + follow its clip (no graph)	555°	72°	2 / 20	10%	0
C: + validated jumps (escape rule, 9.6)	902°	123°	7 / 20	10%	3.7
D: + seek and circle the best loop (9.2–9.5)	**1,385°**	**384°**	**8 / 20**	15%	6.4

Wuji's own pipeline cannot start from a random state, so B (its policy plus plain nearest-frame matching) is the fair baseline here. D beats B in 14 of 20 starts, ties in 3 and is worse in 3. In the 3 ties no validated path into the loop existed, so D did exactly what B did.

FIX The baseline

An earlier message gave Wuji's best as 21.8 s / 1,584°. Those are the reference lengths of two clips that the policy fails in our scene, at 9.5 s and 1.2 s. The fair baseline is the longest clip it completes: `03_continuous_multi_axis/clip_007`, 18.0 s, 1,476°.

⇄ Wuji official vs this project CHANGED

	Wuji official	This project
What "good" means	Track one given clip to its end.	Keep the pen spinning from any start.
Best result (our scene)	18.0 s, 1,476°.	84.7 s, 8,709°.
Weak points	Fixed length and fixed start.	More drops (15% vs 10%), simulation only, no randomization, BC still weak.

Σ Summary 总结

#	Formula	What happened	vs Wuji
1	$\mathbf{p}_t = R(\boldsymbol{\rho}_t^w)^\top(\mathbf{p}_t^{\text{world}} - \mathbf{t}_t^w)$	Clips moved into the wrist frame; velocities and resting flags added.	SAME
2	$\mathbf{f}_t = \mathbf{w} \odot (\mathbf{x}_t - \boldsymbol{\mu})/\boldsymbol{\sigma}$, distances in δ	Every frame became a comparable 67-D point.	NEW
3	$C(u,v) = \frac{1}{\delta}\sum_j \lambda_j \lVert\mathbf{f}_{u+1+j} - \mathbf{f}_{v+j}\rVert \le 6$	101,934 candidate jumps.	NEW
4	$b = \max\left(25, \lceil \Delta q_{\max}/(3\Delta t) - 1\rceil\right)$	600 hold → start transitions; 98 survive.	NEW
5	$b = \text{clip}\left(\lceil \Delta q_{\max}/(\omega_{\max}\Delta t) - 1\rceil, 2, 10\right)$	5,403 removed; every jump smoothed.	NEW
6	$\mathbf{q}^{\text{target}} = \mathbf{q}^{\text{ref}} + \text{EMA}\left(\text{clip}(0.5\,\mathbf{a})\right)$	Wuji's policy, reused unchanged.	SAME
7	$\overline{e}(\mathcal{R}^J) \le 1.5\max\left(\overline{e}(\mathcal{R}^B), 1\,\text{cm}\right)$	38,624 of 40,208 jumps pass.	NEW
8	$c = 25 + 10C$; Dijkstra; SCC; prune	15,215-frame core; 151 loops; best 16.4 s lap.	NEW
9	$v^* = \arg\min_v \lVert\mathbf{f}(s) - \mathbf{f}_v\rVert$; lost if > 3 cm / 40°	Start anywhere, plan into a loop, circle it.	CHANGED
10	$\min_\phi \frac{1}{N}\sum \lVert g_\phi(\hat{\mathbf{o}}) - \widehat{\Delta\mathbf{q}}\rVert^2$	BC trained (weak); PPO export ready.	CHANGED
11	$\Theta = \sum \arccos(\hat{\mathbf{n}}_k \cdot \hat{\mathbf{n}}_{k+1})$	18.0 s → 84.7 s; 1,476° → 8,709°.	CHANGED

💡 The whole project in three sentences

Wuji's policy already knows how to keep a pen on a reference, but each reference is one finite clip. This project changes only the reference: it links frames of all clips into a graph and keeps only the links that Wuji's own policy can execute in simulation. The controller then picks the reference from the hand's state instead of the clock, so it can start anywhere and loop forever.